YOURCOACH

Integration specification

Context

This document describes how to use the YourCoach widget in a “Company” application.
YourCoach widget is a page that should be embedded via iFrame.

Prerequisites

In order to start using YourCoach widget the following steps should be completed:

1. Both **clientId** and **clientSecret** should be requested from YourCoach.
This pair will be used to uniquely identify your application and is required for a handshake flow with YourCoach apis.

2. YourCoach widget requires a token (access/user) which ties current user from “Company” application to YourCoach user session. As for now YourCoach supports only jwt tokens (RFC 7519). In order to trust external token YourCoach application requires to validate token by verifying its signature. See Sequence Flow for more details.

Steps for token generation and sharing:

1. 'Company' needs to create a private/public key pair. See script example here.
2. 'Company' sends YourCoach their public key.
3. During the end-user authorization process 'Company' generates an authorization token containing the user ID using the private key. See script example here.

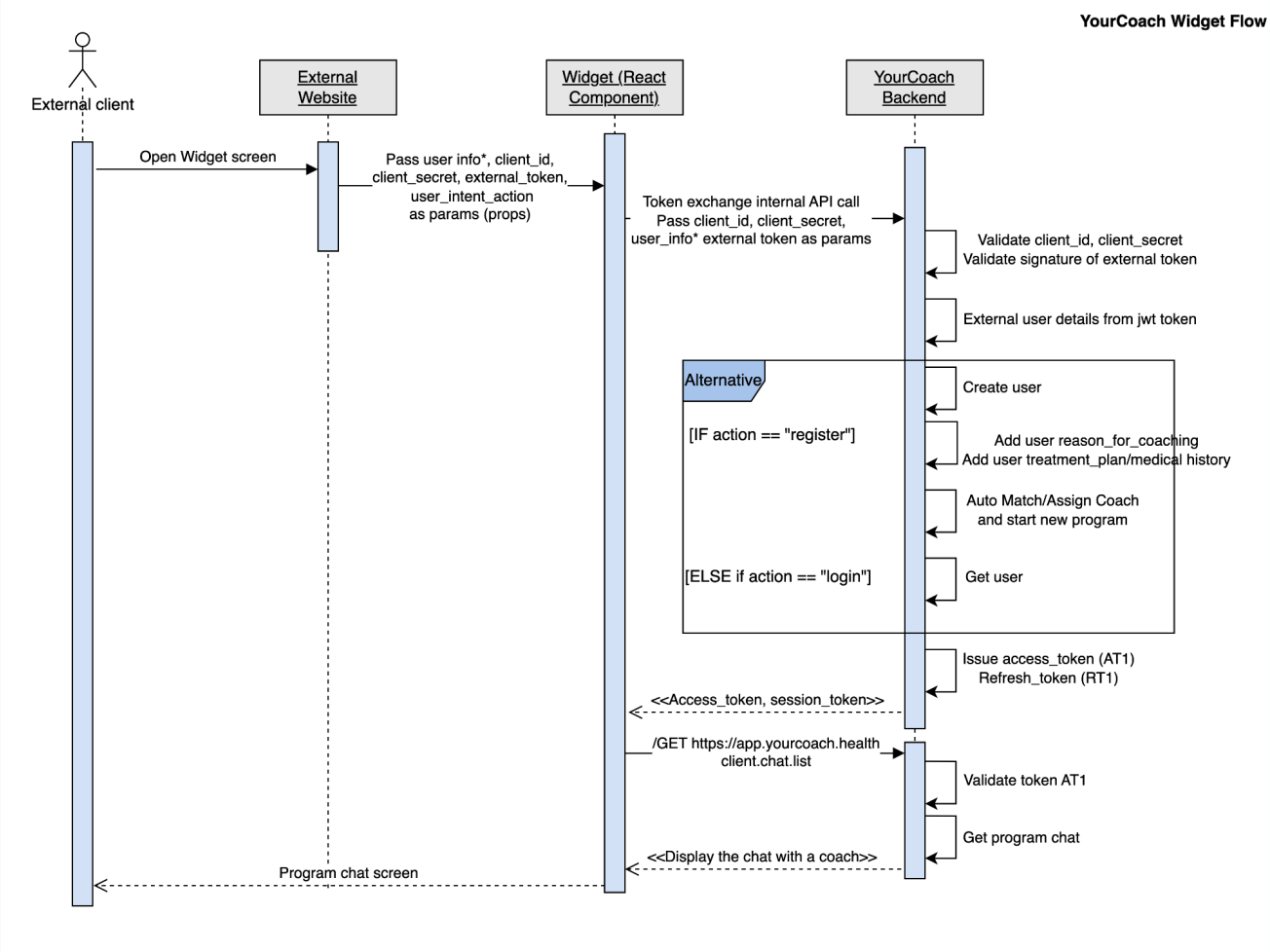
3. Provide YourCoach a page URL where the widget will be embedded.

4. Report SFTP whitelisting + credentials shared and/or S3 bucket whitelisting + bucket name shared:
‘Company’ allows write access to the shared S3 bucket to the following role ARNs:

a) Dev environment (if used) - arn:aws:iam::343506163363:role/api-stage1-ECSTaskRole
b) Staging/Preproduction environment - arn:aws:iam::343506163363:role/api-rc-ECSTaskRole
c) Production environment - arn:aws:iam::343506163363:role/api-prod-ECSTaskRole

Note: *bucket names or SFTP server credentials MUST be unique for every environment used for security reasons*

Sequence flow diagram



The widget uses postMessage API to get the required information for the onboarding process and establish a handshake with the YourCoach API.

Widget usage

A widget is supposed to be used in two action modes: ‘signup’ and ‘login’.

A signup mode should be used when a user is coming from an external website with previously informed consent regarding data sharing. This mode informs the widget that a user registration flow should be started on a YourCoach side.

A login mode could be used from any part of the application (ex. “Company’s” menu) which is a regular flow for an enrolled user. This mode informs the widget that no registration flow is required and should be used when a user is registered on the YourCoach side.

NOTE: the usage of incorrect action mode will lead to an error message displayed on the widget.

Widget Integration

1. Get the right widget page URL based on the environment

```JavaScript
const WIDGET_URL = 'https://widget.yourcoach.health'; // for production env
const WIDGET_URL = 'https://widget.rc.yourcoach.health'; // for staging env
```

2. Render the iframe

```JavaScript
<iframe
  src="{WIDGET_URL}"
  width="100%"
  height="100%"
  style="border: none;"
  id="ych-widget"
  allow="camera;microphone;fullscreen;compute-pressure"
/>
```

NOTE: to make the widget work properly inside a mobile app webview, microphone and camera access should be allowed in the webview. For each platform (iOS/Android) there are different steps to achieve this and it depends on what framework or language is used by the app. If the application supports full-screen mode, make sure that the webview is able to request it as well.

3. Initialize the widget

```JavaScript
const iframe = document.getElementById('ych-widget');

const sendMessageToWidget = data => {
  iframe.contentWindow?.postMessage(data, WIDGET_URL);
};

const initWidget = () => {
  sendMessageToWidget({
    type: 'SIGNUP',
    payload: {
      api: 'core',
      clientId: 'client_id'
      clientSecret: 'client_secret',
      token: 'token',
      appId: 'app_id', // optional
      userInfo: {
        partnerId: 'partner_id',
        populationIds: ['dei_ppl', 'african_american_ppl'], //
optional
        demographics: {
          firstName: 'John',
          lastName: 'Doe',
          email: 'john.doe@example.com',
          phone: '+1234567890', // optional
          state: 'CA', // optional
          zipcode: '12345', // optional
          dateOfBirth: '1990-01-01',
          gender: 'female' | 'male' | 'na' | null
        },
              // HH Treatment Plan, medical or Dtx history (optional)
        additionalData: [
          {
             question: "How ready are you to make changes to your
           health?",
```

```
            answer: "Very"
          }
        ],
      },
    },
  });


// or

  sendMessageToWidget({
    type: 'LOGIN',
    payload: {
      api: 'core',
      clientId: 'client_id'
      clientSecret: 'client_secret',
      token: 'token',
      appId: 'app_id', // optional
    },
  });
};

iframe.onload = function () {
  // need some delay for Safari
  setTimeout(initWidget, 100);
};
```

4. The widget fires some useful events outside that can be used to make a better user experience for the end user in a “Company” application

JavaScript

```
// don't forget to unsubscribe
window.addEventListener('message', event => {
 // make sure that the message is from the widget
 if (event.origin !== WIDGET_URL) {
```

```
    return;
  }

  const {type, payload} = event.data;

  switch (type) {
    case 'TERMS_DECLINED':
      // handle somehow, redirect to another page for example
      break;
    case 'AUTH_ERROR':
      if (payload.error.code === 'USER_DOES_NOT_EXIST') {
        // means that signup flow should be used
      }
      break;
    case 'AUTH_SUCCESS':
      // handle somehow if necessary
      break;
    default:
      break;
  }
});
```

TERMS_DECLINED - fires when a client declined Yourcoach terms of usage
AUTH_ERROR - fires on widget authentication error (payload of the event contains an error)
AUTH_SUCCESS - fires on widget authentication success

NOTE: if there is no way to track on your side that user is registered within YourCoach application then the recommended approach is the following:
1. Try to send the **LOGIN** message first
2. On error received (error.code = 'USER_DOES_NOT_EXIST') fallback to **SIGNUP** message

Parameters

clientId, clientSecret - pair will be provided by YourCoach see Prerequisites section for details
token - External token which allows YourCoach to identify current user
appId - to be used for multiple upstream platforms
userInfo - a json object which represents user data information, example:

```JavaScript
interface UserInfo {
  partnerId: string;
  demographics: {
    firstName: string;
    lastName: string;
    email: string;
    phone: string | null;
    state: string | null;
    zipcode: string | null;
    dateOfBirth: string;
    gender: 'female' | 'male' | 'na' | null
  };
  // HH Treatment Plan, medical or Dtx history
  additionalData: {
    question: string,
    answer: string | string[];
  }[];
}
```

In case of “signup” flow a userInfo param is required and the widget will show an error if it’s missing. Also errors will occur in case an attempt to re-register the user that already exists in the YourCoach system.

User auth API endpoint

In order to register the user and start the program without opening the widget you can also call the auth API directly (`"user.auth.external_auth"`). Parameters are almost the same as in the widget registration case but using the “snake_case” instead of “camelCase” in their names.

Additional params:
action: string - “register” (it can also be “login” but it’s not suitable for the case described)
device: string - “any device info like user agent string”

URLs
Preproduction (staging env) - https://api.rc.yourcoach.health

Development staging env - https://api.stage1.yourcoach.health

Request example:

JavaScript

```
{
    "id": 14,
    "meta": {
        "version": [10, 0]
    },
    "method": "user.auth.external_auth",
    "params": {
        "client_id": "test_client_id",
        "client_secret": "AM30Ox9zULYfBLQTRIcC",
        "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ1c2VyX2lkIjo … u_UKqPkI",
        "action": "register",
        "device": "Some device or browser name",
        "push_token": null,  // not used
       "population_ids": ['dei_ppl', 'african_american_ppl'], // optional
        "app_id": "some_app_id",
        "user_info":     {
          "partner_id": "19",
          "demographics": {
            "first_name": "John",
            "last_name": "Doe", // optional
            "email": "johndoe@example.com",
            "phone": null, // optional
            "state": "CA", // optional
            "zipcode": "12345", // optional
            "date_of_birth": null, // optional
            "gender": "female" // optional but highly appreciated
          },
          "additional_data": [
               {
                 "question": "How ready are you to make changes to your health?",
                 "answer": "Very"
               },
               {
                 "question": "Please share any health-related concerns or additional
              details that you would like to share with your coach. If none, just
              write \"none\".",
                 "answer": "None"
               },
```

```
                {
                  "question": "Before I can connect you with a coach, I need your
              permission to share your information along with your agreement to the
              coaching terms and conditions. Let’s review the health disclosure
              first.",
                  "answer": "Sounds good, let's review"
                }
            ]
        }
    }
}
```

Real-time Program Status API end-point

Provides a real-time coaching program status (new user, in-progress, extended, archived).

URLs
Stage - https://api.rc.yourcoach.health

Request example:

JavaScript

```
{
    "id": 4,
    "meta": {
        "version": [10, 0]
    },
    "method": "user.auth.check_status",
    "params": {
        "client_id": "test_client_id",
        "client_secret": "AM30Ox9zULYfBLQTRIcC",
        "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ1c2VyX2lkIjo … u_UKqPkI",
        "_response": "renewed",
    }
}
```

Params:

client_id: string - internal client id
client_secret: string - client secret
token: string - token containing `user_id` in client's system as payload

Response:

status: user status to choose access to the Widget (which flow to run - client onboarding, direct access to the widget, to collect the client feedback, etc…)

Error response example:

```javascript
{
  "id": 4,
  "meta": {},
  "result": null,
  "error": {
    "code": "error.resource.invalid_context",
    "message": "Invalid context",
    "data": {
      "reason": "The client_id/client_secret combination is invalid",
      "method": "user.auth.check_status",
      "version": [10, 0]
    }
  },
  "warnings": []
}
```

Eligibility events API end-point

Receives user's eligibility for coaching events and makes the corresponding actions (stops/restarts program or changes its auto extension status).

URLs
Preproduction (staging env) - https://api.rc.yourcoach.health
Development staging env - https://api.stage1.yourcoach.health

Request example:

```JavaScript
{
    "id": 5,
    "meta": {
        "version": [10, 0]
    },
    "method": "user.hooks.eligibility",
    "params": {
        "client_id": "test_client_id",
        "client_secret": "AM30Ox9zUfBLQTRIcC",
        "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ1c2VyX2lkIjo … u_UKqPkI",
        "event": {
           "eligibilty": "stop_now" | "stop_renewal" | "restart_now" |
"restart_renewal",
           // datetime ISO string, optional, default is now
           "generated_at": "2024-02-02T12:00:00Z",
           // human readable description, optional
           "description": "User canceled the subscription",
           // eligibility change reason code, subject to discuss
           "reason": "subscription_canceled" | "failed_payment" | "refund_requested"
| "subscription_restarted" | "user_fired" | "fraud_detected" | "refunded" |
"account_deleted" | "trial_expired"

           // additional params possible, subject to discuss
        }
    }
}
```

Params:
client_id: string - internal client id
client_secret: string - client secret
token: string - authorization token containing `user_id` in the client's system as a payload
event.eligibility: the type of the eligibility event, mandatory
event.generated_at: the time when the event happened, optional, default is now
event.description: human-readable description of the event, default is empty
event.reason: reason code of the event happened
At the MVP stage API will accept an arbitrary additional set of params inside the event object that you may consider useful

Response:
Nothing (code 204) but it's a subject to discuss and to be changed in the future
Raises `"error.resource.invalid_context"` if the event cannot be processed in the current context (e.g. attempt to restart already started program)

User profile update API end-point

Receives user's eligibility for coaching events and makes the corresponding actions (stops/restarts program or changes its auto extension status).

URLs
Preproduction (staging env) - https://api.rc.yourcoach.health
Development staging env - https://api.stage1.yourcoach.health

Request example:

JavaScript

```
{
    "id": 14,
    "meta": {
        "version": [10, 0]
    },
    "method": "user.hooks.update_profile",
    "params": {
        "client_id": "test_client_id",
        "client_secret": "AM30Ox9zULYfBLQTRIcC",
        "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJ1c2VyX2lkIjo … u_UKqPkI",
        "demographics": {
            "first_name": "John", // optional
            "last_name": "Doe", // optional
            "email": "johndoe@example.com", // optional
            "phone": null, // optional
            "state": "CA", // optional
            "zipcode": "12345", // optional
            "date_of_birth": null, // optional
            "gender": "female" // optional         },
    }
}
```

Params:
client_id: string - internal client id
client_secret: string - client secret
token: string - authorization token containing `user_id` in the client's system as a payload
demographics: same object as with registering user but all parameters are optional

Response:
Nothing (code 204) but it's a subject to discuss and to be changed in the future

Notification API requirements

In order to receive push notifications for your users, you must implement the following endpoint and provide us with the URL, auth header name, and auth header value (API key/token).
It should accept a POST request in the following format (JSON):

JavaScript

```
// headers
"Some-Authorization-Header-Name": "secret_api_key"
"Content-Type": "application/json"

// request body
{
    // list of your app ids to sent notifications to (may be ignored), see the widget
integration API
    "app_ids": ["app1_id", "app2_id"],
    "events": [  // list of events to sent to the specified users
        {
            "user_id": "9876543",  // your internal user ID to sent notification

             // the type of the event to send, see below (may be ignored)
            "name": "yc_event_type",

            // datetime of the event in ISO string format, "%Y-%m-%dT%H:%M:%S"
            // (may be ignored), UTC timezone
            "time": "2023-01-20T12:00:05",

            "properties": {
                // Actual notification title and body to be sent to the user
                // May change dynamically even within the same event type
                "title": "Message title",
                "message": "Message body",
            },
```

```
        },
    ]
}
```

The list of supporting push-notifications:

Case	Event name
Session scheduled	yc_conference_scheduled
Session rescheduled	yc_conference_resheduled
Session canceled	yc_user_canceled_session
Coach writes to the chat	yc_post_created
Session reminder (24 hours)	yc_conference_upcoming_24hr
Session reminder (1 hour)	yc_conference_upcoming_1hr
Session reminder (session is starting)	yc_conference_starting
Session started reminder (5min)	yc_conference_reminder
Re-engage if no user action (10 d)	yc_membership_10d_no_activity
Re-engage if no user action (20 d)	yc_membership_20d_no_activity
Re-engage if no user action after session (5 d)	yc_conference_5d_no_activity
Renewal (extend program)	yc_course_extended

Reports Generated

	Report name	Frequency (report is generated)	Brief description
1	program_pair	Daily	Shows basic info about the program status and coach match
2	sessions	Daily	Shows details about live sessions
3	chat_transcript_data	Daily	Chat messages separated by roles (dependent on business contract)
4	course_restart_dates	Daily	Shows all programs that were restarted (extended) on that day
5	coaching_engagement	Weekly	Sessions scheduled, missed, attended. Messages exchanged total by Coach, by User
6	coaching_program_summary	Weekly	Program info: users, sessions, program length
7	client_billing_reconciliation	Monthly	Overview of Coaching Program Activity (new enrollments, re-enrollments, and live sessions)
8	sessions_and_program_cycles	Monthly	Shows program cycles and the sessions within them

Additional attachments

Public/private key pair generation script

JavaScript

```
#!/usr/bin/env node
/* eslint-disable no-console */

const crypto = require("crypto");
const fs = require("fs");
const path = require("path");

const keyName = process.argv[2];
```

```
async function generateKeyPair() {
 try {
  const { publicKey, privateKey } = crypto.generateKeyPairSync("rsa", {
   modulusLength: 2048,
   publicKeyEncoding: {
    type: "spki",
    format: "pem",
   },
   privateKeyEncoding: {
    type: "pkcs8",
    format: "pem",
   },
  });

  const KEYS_PATH = "./keys";

  fs.existsSync(path.resolve(KEYS_PATH)) &&
   fs.rmdirSync(path.resolve(KEYS_PATH), { recursive: true });

  fs.mkdirSync(path.resolve(KEYS_PATH), { recursive: true });

  // save keys to file using tenant name
  fs.writeFileSync(
   path.resolve(`${KEYS_PATH}/${keyName}.public.pem`),
   publicKey
  );
  fs.writeFileSync(
   path.resolve(`${KEYS_PATH}/${keyName}.private.pem`),
   privateKey
  );

  console.log(
   `✅ ${keyName}.public.pem and ${keyName}.private.pem have been successfully
generated!`
  );
 } catch (err) {
  console.error(err);
 }
}

generateKeyPair();
```

Token generation example script

```JavaScript
import * as jose from "jose";

const PRIVATE_KEY = `-----BEGIN PRIVATE KEY-----
MIIEvgIBADANBgkqhkiG9w0BAQEFAASCBKgwggSkAgEAAoIBAQDpWPDVuo0wriaC
.....
.....
.....
.....
/ZdsbaxR1rhd5Gl4QEf2FQBq
-----END PRIVATE KEY-----`;

export const generateAuthToken = async (userId: string) => {
 try {
  const alg = "RS256";

  const privateKey = await jose.importPKCS8(PRIVATE_KEY, alg);

  const jwt = await new jose.SignJWT({ user_id: userId })
   .setProtectedHeader({ alg })
   .setIssuedAt()
   .setIssuer("Your Company")
   .setExpirationTime("1m")
   .sign(privateKey);

  return jwt;
 } catch (error) {
  throw error;
 }
};
```

Mobile-specific (iOS, Android) additional configurations

iOS

1. Create a **WKWebViewConfiguration** object and enable media capture and playback:

```JavaScript
let webConfiguration = WKWebViewConfiguration()
```

```
webConfiguration.allowsInlineMediaPlayback = true
webConfiguration.mediaTypesRequiringUserActionForPlayback = []

// pay attention that webView.configuration.property syntax won't work
// the configuration instance must be created before the web view
let webView = WKWebView(frame: .zero, configuration: webConfiguration)
```

2. Disable **scroll**

JavaScript

```
webView.scrollView.isScrollEnabled = false
```

3. Disable **pinch to zoom** gesture

JavaScript

```
func scrollViewWillBeginZooming(_ scrollView: UIScrollView, with view: UIView?) {
 if parent.needRemoveZoom {
  zoomScale = scrollView.zoomScale
  scrollView.pinchGestureRecognizer?.isEnabled = false
 }
}

func scrollViewDidZoom(_ scrollView: UIScrollView) {
 if let originalScale = zoomScale {
  scrollView.zoomScale = originalScale
 }
}
```

4. Disable **adjustContentInset**

```JavaScript
NotificationCenter.default.removeObserver(webView, name:
UIResponder.keyboardWillChangeFrameNotification, object: nil)
NotificationCenter.default.removeObserver(webView, name:
UIResponder.keyboardWillShowNotification, object: nil)
NotificationCenter.default.removeObserver(webView, name:
UIResponder.keyboardWillHideNotification, object: nil)
```

5. Add the necessary permissions to the **Info.plist** file

```JavaScript
<key>NSCameraUsageDescription</key>
<string>Your camera usage description</string>
<key>NSMicrophoneUsageDescription</key>
<string>Your microphone usage description</string>
```

6. Set up opening **URLs** on a new screen

```JavaScript
func webView(_ webView: WKWebView, createWebViewWith configuration:
WKWebViewConfiguration, for navigationAction: WKNavigationAction, windowFeatures:
WKWindowFeatures) -> WKWebView? {
  if navigationAction.targetFrame == nil, let url = navigationAction.request.url {
   // openUrlOnANewScreen(url)
  }
  return nil
 }
```

Android

1. Add the following configuration for the **web view**

```JavaScript
WebView myWebView = (WebView) findViewById(R.id.webview);
WebSettings webSettings = myWebView.getSettings();

webSettings.javaScriptEnabled = true
webSettings.setSupportZoom(false)
webSettings.allowFileAccess = true
webSettings.domStorageEnabled = true
webSettings.mediaPlaybackRequiresUserGesture = false
```

2. Add the necessary permissions to **AndroidManifest.xml** file

```JavaScript
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-permission android:name="android.permission.MODIFY_AUDIO_SETTINGS" />
```

3. Set up opening **URLs** on a new screen

```JavaScript
webView.setWebChromeClient(new WebChromeClient() {
  @Override
  public boolean onCreateWindow(WebView view, boolean isDialog, boolean isUserGesture,
Message resultMsg) {
    WebView.HitTestResult result = view.getHitTestResult();
    String url = result.getExtra();

    Intent intent = new Intent(view.getContext(), NewWebViewActivity.class);
    intent.putExtra("url", url);
    view.getContext().startActivity(intent);

    return false; // Indicates that we handled the URL
  }
});
```

4. Allow to pick **files** from a device

```JavaScript
webView.setWebChromeClient(new WebChromeClient() {
  // For Android 5.0+
  public boolean onShowFileChooser(WebView view, ValueCallback<Uri[]> filePath,
WebChromeClient.FileChooserParams fileChooserParams) {
    // Double check that we don't have any existing callbacks
    if (mFilePathCallback != null) {
      mFilePathCallback.onReceiveValue(null);
    }
    mFilePathCallback = filePath;

    // Get the accepted MIME types
    String[] acceptTypes = fileChooserParams.getAcceptTypes();

    Intent intent = fileChooserParams.createIntent();
    intent.setType(types); // Set the MIME type here

    try {
      startActivityForResult(intent, REQUEST_CODE);
    } catch (ActivityNotFoundException e) {
      mFilePathCallback = null;
      Toast.makeText(getApplicationContext(), "Cannot open file chooser",
Toast.LENGTH_LONG).show();
      return false;
    }

    return true;
  }
});
```

We currently support only the following **MIME** types:

- image/jpeg
- image/png
- text/plain
- application/msword
- application/vnd.openxmlformats-officedocument.wordprocessingml.document
- application/vnd.ms-excel
- application/vnd.openxmlformats-officedocument.spreadsheetml.sheet
- application/vnd.ms-powerpoint
- application/vnd.openxmlformats-officedocument.presentationml.presentation
- application/pdf